

Token-by-Token Training and Stochastic Gradient Descent (SGD) in Language Modeling

Introduction

In language modeling, training is often done **token by token**, where the model processes input sequences one token at a time and predicts the next token in the sequence. This approach is commonly used in tasks like text generation, machine translation, and speech recognition. To optimize the model, **Stochastic Gradient Descent (SGD)** is used, where the gradient of the loss is approximated using a single sequence (or a mini-batch of sequences) from the corpus.

Token-by-Token Training

1. Tokenization

The input text is split into tokens (e.g., words, subwords, or characters). For example, the sentence "I love machine learning" might be tokenized as:

$$\text{Tokens} = [\text{I}, \text{love}, \text{machine}, \text{learning}]$$

Each token is mapped to a unique integer ID using a vocabulary:

$$\text{Token IDs} = [5, 12, 27, 43]$$

2. Model Input

The token IDs are fed into the model as a sequence. For a sequence of length T , the input is:

$$\mathbf{x} = [x_1, x_2, \dots, x_T]$$

where x_t is the token ID at position t .

3. Embedding Layer

Each token ID is converted into a dense vector representation using an embedding layer:

$$\mathbf{E} \in \mathbb{R}^{V \times d}$$

where: - V is the vocabulary size. - d is the embedding dimension.

The input sequence $\mathbf{x}$ is transformed into a sequence of embeddings:

$$\mathbf{h}_t = \mathbf{E}[x_t] \quad \text{for } t = 1, 2, \dots, T$$

where $\mathbf{h}_t \in \mathbb{R}^d$ is the embedding of the token at position t .

4. Sequential Processing

The sequence of embeddings $[\mathbf{h}_1, \mathbf{h}_2, \dots, \mathbf{h}_T]$ is processed by the model (e.g., an RNN, LSTM, or Transformer). For example, in an RNN, the hidden state $\mathbf{s}_t$ at time step t is computed as:

$$\mathbf{s}_t = f(\mathbf{h}_t, \mathbf{s}_{t-1})$$

where f is the recurrent function.

5. Output Prediction

At each time step t , the model predicts the next token y_t (or a probability distribution over the vocabulary):

$$\mathbf{o}_t = g(\mathbf{s}_t)$$

where: - $\mathbf{o}_t \in \mathbb{R}^V$ is the output logits (unnormalized scores) for the vocabulary. - g is the output function (e.g., a fully connected layer followed by a softmax).

The predicted token $\hat{y}_t$ is:

$$\hat{y}_t = \arg \max(\text{softmax}(\mathbf{o}_t))$$

6. Loss Calculation

The model is trained to minimize the **cross-entropy loss** between the predicted token $\hat{y}_t$ and the true token y_t at each time step:

$$\mathcal{L}_t = -\log P(y_t | y_1, y_2, \dots, y_{t-1})$$

where $P(y_t | y_1, y_2, \dots, y_{t-1})$ is the probability of the true token y_t given the previous tokens.

The total loss for the sequence is the average loss over all time steps:

$$\mathcal{L} = \frac{1}{T} \sum_{t=1}^T \mathcal{L}_t$$

—

Stochastic Gradient Descent (SGD)

1. Corpus-Level Loss

A **corpus** is a collection of sequences (e.g., sentences or documents). Let the corpus contain N sequences:

$$\mathcal{D} = \{\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots, \mathbf{x}^{(N)}\}$$

For each sequence $\mathbf{x}^{(i)} = [x_1^{(i)}, x_2^{(i)}, \dots, x_{T_i}^{(i)}]$, the loss is computed as:

$$\mathcal{L}^{(i)} = \frac{1}{T_i} \sum_{t=1}^{T_i} \mathcal{L}_t^{(i)}$$

where: - $\mathcal{L}_t^{(i)} = -\log P(y_t^{(i)} | y_1^{(i)}, y_2^{(i)}, \dots, y_{t-1}^{(i)})$ is the loss at time step t for sequence i . - T_i is the length of sequence i .

The **total loss** over the entire corpus is:

$$\mathcal{L}_{\text{corpus}} = \frac{1}{N} \sum_{i=1}^N \mathcal{L}^{(i)}$$

2. SGD Steps

Instead of computing the gradient of the total loss over the entire corpus, SGD approximates it using a **single sequence** (or a mini-batch of sequences). The steps are:

1. **Sample a Sequence**: - Randomly select a sequence $\mathbf{x}^{(i)}$ from the corpus $\mathcal{D}$.
2. **Compute Sequence Loss**: - Compute the loss $\mathcal{L}^{(i)}$ for the selected sequence:

$$\mathcal{L}^{(i)} = \frac{1}{T_i} \sum_{t=1}^{T_i} \mathcal{L}_t^{(i)}$$

3. **Compute Gradient**: - Compute the gradient of the loss $\mathcal{L}^{(i)}$ with respect to the model parameters $\mathbf{W}$:

$$\nabla_{\mathbf{W}} \mathcal{L}^{(i)} = \frac{1}{T_i} \sum_{t=1}^{T_i} \nabla_{\mathbf{W}} \mathcal{L}_t^{(i)}$$

4. ****Update Parameters****: - Update the model parameters using the gradient:

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \nabla_{\mathbf{W}} \mathcal{L}^{(i)}$$

where η is the learning rate.

Example: "I love machine learning"

Tokenization

- Input text: "I love machine learning" - Tokenized: [I, love, machine, learning] - Token IDs: [5, 12, 27, 43]

Sequence Loss

For the sequence $\mathbf{x} = [5, 12, 27, 43]$, the loss is:

$$\mathcal{L} = \frac{1}{4} (\mathcal{L}_1 + \mathcal{L}_2 + \mathcal{L}_3 + \mathcal{L}_4)$$

where:

$$\mathcal{L}_1 = -\log P(12 \mid 5)$$

$$\mathcal{L}_2 = -\log P(27 \mid 5, 12)$$

$$\mathcal{L}_3 = -\log P(43 \mid 5, 12, 27)$$

$$\mathcal{L}_4 = -\log P(\text{EOS}_i \mid 5, 12, 27, 43)$$

SGD Step

1. ****Sample a Sequence****: - Suppose we sample $\mathbf{x} = [5, 12, 27, 43]$.
2. ****Compute Sequence Loss****:

$$\mathcal{L} = \frac{1}{4} (\mathcal{L}_1 + \mathcal{L}_2 + \mathcal{L}_3 + \mathcal{L}_4)$$

3. ****Compute Gradient****:

$$\nabla_{\mathbf{W}} \mathcal{L} = \frac{1}{4} (\nabla_{\mathbf{W}} \mathcal{L}_1 + \nabla_{\mathbf{W}} \mathcal{L}_2 + \nabla_{\mathbf{W}} \mathcal{L}_3 + \nabla_{\mathbf{W}} \mathcal{L}_4)$$

4. ****Update Parameters****:

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \nabla_{\mathbf{W}} \mathcal{L}$$

Key Takeaways

- Token-by-token training involves processing input sequences one token at a time and predicting the next token. - The loss for a sequence is the average loss over all tokens in that sequence. - SGD approximates the gradient of the corpus-level loss using a single sequence (or mini-batch) and updates the model parameters. - This approach is computationally efficient and helps the model converge faster.
